Dissertation: Autonomous Agent for Alpha Discovery in Indian Equity Markets

Author: Abhinav Gazta
Course: S1-25_AIMLCZG628T (Dissertation)
Topic: Financial Machine Learning & Deep Reinforcement Learning (DRL)

---

1. Abstract & Objectives

This notebook implements a state-of-the-art **Deep Reinforcement Learning (DRL)** framework to optimize portfolio allocation on the NSE/BSE. Unlike traditional backtests, this solution addresses **Non-Stationarity** and **Look-Ahead Bias** using **Walk-Forward Validation** and **Bayesian Hyperparameter Tuning**.

Key Technical Components:

1. **Hybrid Transformer-PPO Architecture:** Uses Self-Attention to capture long-term market regimes.
2. **Dynamic Cost Simulator:** Models Linear (STT/Brokerage) and Quadratic (Impact Cost) frictions.
3. **Bayesian Optimization (Optuna):** Automatically tunes Learning Rate, Entropy, and Reward Penalties.
4. **Walk-Forward Validation:** A rolling-window evaluation to ensure the agent adapts to changing markets.
5. **Explainable AI (XAI):** Uses SHAP values to interpret the "Black Box" decision-making process.
6. **Statistical Rigor:** Monte Carlo Permutation tests to validate if Alpha is statistically significant ($p < 0.05$).

In [83]:
```
import gymnasium as gym
from gymnasium import spaces
import pandas as pd
import numpy as np
import yfinance as yf
import torch
import torch.nn as nn
import torch.optim as optim
from stable_baselines3 import PPO
from stable_baselines3.common.torch_layers import BaseFeaturesExtractor
import ta
import matplotlib.pyplot as plt
import seaborn as sns
import shap
```

```
import optuna
import scipy.stats as stats
import warnings

# Suppress warnings for cleaner output
warnings.filterwarnings("ignore")

# Use GPU if available
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Using device: {device}")
```

```
Using device: cpu
```

In [84]:

```
# ==========================================
# 1.1 VISUALIZATION MODULE (EDA) - FIXED
# ==========================================
import matplotlib.pyplot as plt
import matplotlib.dates as mdates

def visualize_market_dynamics(ticker, start_date="2020-01-01", end_date="202
    """
    Generates a professional financial dashboard for Dissertation EDA.
    Visualizes Price, Volatility (BB), Momentum (RSI), and Trend (MACD).
    """
    print(f"\n--- Generating Market Regime Analysis for {ticker} ---")

    # 1. Fetch RAW Data
    # auto_adjust=True handles splits/dividends
    df = yf.download(ticker, start=start_date, end=end_date, auto_adjust=Tru

    if df.empty:
        print("Error: No data found for ticker.")
        return

    # --- FIX: Flatten MultiIndex columns if present ---
    # This prevents the (N, 1) dimension error
    if isinstance(df.columns, pd.MultiIndex):
        df.columns = df.columns.get_level_values(0)

    # --- FIX: Ensure 'Close' is a 1D Series ---
    # use .squeeze() to convert single-column DataFrame to Series
    close_series = df['Close'].squeeze()

    # 2. Compute Indicators (Matching Agent's Logic)
    # Bollinger Bands (Volatility)
    # Pass the 1D series explicitly
    bb = ta.volatility.BollingerBands(close_series)
    df['bb_high'] = bb.bollinger_hband()
    df['bb_low'] = bb.bollinger_lband()

    # RSI (Momentum)
    df['rsi'] = ta.momentum.RSIIndicator(close_series, window=14).rsi()

    # MACD (Trend)
    macd_ind = ta.trend.MACD(close_series)
    df['macd'] = macd_ind.macd()
```

```
    df['macd_signal'] = macd_ind.macd_signal()
    df['macd_hist'] = macd_ind.macd_diff()

    # 3. Create Dashboard (4 Subplots)
    fig, axes = plt.subplots(4, 1, figsize=(16, 12), sharex=True,
                             gridspec_kw={'height_ratios': [3, 1, 1, 1], 'hs

    # Plot A: Price & Volatility Regimes
    axes[0].plot(df.index, df['Close'], label='Close Price', color='black',
    axes[0].plot(df.index, df['bb_high'], label='Upper BB (2std)', color='gr
    axes[0].plot(df.index, df['bb_low'], label='Lower BB (2std)', color='red
    axes[0].fill_between(df.index, df['bb_high'], df['bb_low'], color='gray'
    axes[0].set_title(f"{ticker}: Market Regimes & Technical Features", font
    axes[0].set_ylabel("Price (INR)")
    axes[0].legend(loc='upper left')
    axes[0].grid(True, alpha=0.3)

    # Plot B: Volume (Liquidity)
    # Handle Volume being Series or DataFrame
    vol = df['Volume'].squeeze()
    colors = ['green' if o < c else 'red' for o, c in zip(df['Open'], df['Cl
    axes[1].bar(df.index, vol, color=colors, alpha=0.7)
    axes[1].set_ylabel("Volume")
    axes[1].set_title("Liquidity Profile", fontsize=10)
    axes[1].grid(True, alpha=0.3)

    # Plot C: RSI (Momentum)
    axes[2].plot(df.index, df['rsi'], color='purple', label='RSI (14)')
    axes[2].axhline(70, color='red', linestyle='--', linewidth=0.8)
    axes[2].axhline(30, color='green', linestyle='--', linewidth=0.8)
    axes[2].fill_between(df.index, 70, 30, color='purple', alpha=0.05)
    axes[2].set_ylabel("RSI")
    axes[2].set_ylim(0, 100)
    axes[2].legend(loc='upper left', fontsize=8)
    axes[2].grid(True, alpha=0.3)

    # Plot D: MACD (Trend)
    axes[3].plot(df.index, df['macd'], label='MACD Line', color='blue', line
    axes[3].plot(df.index, df['macd_signal'], label='Signal Line', color='or
    axes[3].bar(df.index, df['macd_hist'], label='Histogram', color='gray',
    axes[3].set_ylabel("MACD")
    axes[3].axhline(0, color='black', linewidth=0.8)
    axes[3].legend(loc='upper left', fontsize=8)
    axes[3].grid(True, alpha=0.3)

    # Formatting
    axes[3].xaxis.set_major_formatter(mdates.DateFormatter('%Y-%m'))
    plt.xlabel("Date")
    plt.tight_layout()
    plt.show()

# Run the fixed function
visualize_market_dynamics('RELIANCE.NS', start_date="2020-01-01", end_date="
visualize_market_dynamics('INFY.NS', start_date="2020-01-01", end_date="2024
visualize_market_dynamics('HDFCBANK.NS', start_date="2020-01-01", end_date="
visualize_market_dynamics('TCS.NS', start_date="2020-01-01", end_date="2024-
```

--- Generating Market Regime Analysis for RELIANCE.NS ---

--- Generating Market Regime Analysis for INFY.NS ---

--- Generating Market Regime Analysis for HDFCBANK.NS ---

```
--- Generating Market Regime Analysis for TCS.NS ---
```

2. Advanced Financial Metrics Library

To validate the agent's performance, we implement a robust metrics library. This includes the **Probabilistic Sharpe Ratio (PSR)**, which adjusts the standard Sharpe Ratio for skewness and kurtosis, providing a confidence level that the strategy is truly profitable.

In [85]:
```python
class FinancialMetrics:
    @staticmethod
    def get_max_drawdown(prices):
        prices = np.array(prices)
        if len(prices) < 2: return 0.0
        peaks = np.maximum.accumulate(prices)
        # Avoid division by zero
        with np.errstate(divide='ignore', invalid='ignore'):
            drawdowns = (prices - peaks) / (peaks + 1e-9)
        return np.min(np.nan_to_num(drawdowns))

    @staticmethod
    def get_sharpe_ratio(returns, risk_free_rate=0.06):
        returns = np.nan_to_num(returns)
        if len(returns) < 2: return 0.0

        # Annualized Excess Return
```

```
        excess_ret = returns - (risk_free_rate / 252)
        std_dev = np.std(excess_ret)

        if std_dev < 1e-9: return 0.0

        # Annualize Sharpe
        return (np.mean(excess_ret) / std_dev) * np.sqrt(252)

    @staticmethod
    def get_sortino_ratio(returns, risk_free_rate=0.06):
        returns = np.nan_to_num(returns)
        excess_ret = returns - (risk_free_rate / 252)
        downside_returns = excess_ret[excess_ret < 0]

        downside_std = np.std(downside_returns)
        if downside_std < 1e-9: return 0.0

        return (np.mean(excess_ret) / downside_std) * np.sqrt(252)

    @staticmethod
    def probabilistic_sharpe_ratio(returns, benchmark_sharpe=0):
        """
        Calculates the probability that the strategy's Sharpe Ratio is great
        Adjusts for non-normal distribution (Skew/Kurtosis).
        """
        returns = np.nan_to_num(returns)
        sr = FinancialMetrics.get_sharpe_ratio(returns)
        skew = stats.skew(returns)
        kurtosis = stats.kurtosis(returns)
        n = len(returns)

        if n < 2: return 0.0

        sr_std = np.sqrt((1 + (0.5 * skew * sr) + ((kurtosis - 3) / 4) * sr*
        if sr_std < 1e-9: return 0.0

        z_stat = (sr - benchmark_sharpe) / sr_std
        return stats.norm.cdf(z_stat)
```

3. Data Engineering Layer

This layer handles the extraction and preprocessing of OHLCV data. **Crucial Step:** We engineer features that capture **Momentum** (RSI), **Trend** (MACD), **Volatility** (Bollinger Width), and **Liquidity** (Volume Ratio). All features are normalized using a rolling Z-score to ensure stationarity, which is critical for Neural Network stability.

3.1 Enhanced Data Pipeline: Local CSV + Cloud Integration

Important Update: This notebook now supports two complementary data sources to maximize training dataset size:

Data Sources:

1. **Local CSV Dataset** (`data/raw/ohlc-data-10yrs/`):
    - Contains 10 years of historical OHLCV data for 50+ NSE/BSE stocks
    - Enables training on 2,500+ trading days per stock
    - Immediate availability without network latency
2. **Cloud Fallback (yfinance)**:
    - Supplements recent data and missing tickers
    - Optional real-time updates
    - Automatic fallback if local data unavailable

Hybrid Loading Strategy:

- **Phase 1**: Automatically loads all available stocks from local CSV directory
- **Phase 2** (Optional): Fetches additional/recent data from yfinance
- **Phase 3**: Merges datasets with conflict resolution (online data has priority for recent dates)

Dataset Size Comparison:

Data Source	Stocks	Date Range	Total Rows
Online Only (4 tickers, 10 years)	4	2015-2025	~10,000
Local CSV (50 stocks, 10+ years)	50+	2012-2024	125,000+
Combined (Best of both)	50+	2012-2025	130,000+

This vastly larger dataset enables the DRL agent to:

- Learn from diverse market regimes
- Better generalize to unseen market conditions
- Achieve higher statistical significance in performance metrics
- Reduce look-ahead bias through longer validation windows

In [86]:
```
def fetch_and_process_data(tickers, start_date="2015-01-01", end_date="2025-
    print(f"Fetching data for {len(tickers)} assets...")
    data = yf.download(tickers, start=start_date, end=end_date, group_by='ti
    processed_frames = []

    for ticker in tickers:
        if len(tickers) == 1:
            df = data.copy()
        else:
            try:
                df = data[ticker].copy()
            except KeyError:
```

```
                continue

        if df.empty: continue

        # --- Feature Engineering ---
        # 1. Log Returns: Stationarity
        df['log_ret'] = np.log(df['Close'] / (df['Close'].shift(1) + 1e-8))

        # 2. Momentum & Trend
        df['rsi'] = ta.momentum.RSIIndicator(df['Close'], window=14).rsi() /
        macd = ta.trend.MACD(df['Close'])
        df['macd'] = macd.macd_diff()

        # 3. Volatility
        bb = ta.volatility.BollingerBands(df['Close'])
        df['bb_width'] = (bb.bollinger_hband() - bb.bollinger_lband()) / (df
        df['atr'] = ta.volatility.AverageTrueRange(df['High'], df['Low'], df

        # 4. Liquidity / Cost Proxies
        df['adv_20'] = df['Close'] * df['Volume'].rolling(window=20).mean()
        df['vol_ratio'] = (df['Close'] * df['Volume']) / (df['adv_20'] + 1e-

        df.dropna(inplace=True)

        # 5. Normalization (Rolling Z-Score)
        cols_to_norm = ['log_ret', 'rsi', 'macd', 'bb_width', 'vol_ratio']
        for col in cols_to_norm:
            rolling_mean = df[col].rolling(60).mean()
            rolling_std = df[col].rolling(60).std()
            df[col] = (df[col] - rolling_mean) / (rolling_std + 1e-8)

        df['ticker'] = ticker
        processed_frames.append(df.dropna())

    if not processed_frames:
        raise ValueError("No data downloaded.")

    combined = pd.concat(processed_frames)
    pivot_df = combined.pivot_table(index='Date', columns='ticker', values=[

    # Clean Infinite/NaN values
    pivot_df.replace([np.inf, -np.inf], np.nan, inplace=True)
    pivot_df.dropna(inplace=True)

    return pivot_df
```

In [87]:

```
import os
import glob

def load_local_csv_dataset(data_dir):
    """
    Load OHLCV data from local CSV files in the data/raw/ohlc-data-10yrs dir
    This enables training on a much larger dataset (10 years of NSE/BSE data

    Args:
        data_dir: Path to directory containing individual stock CSV files
```

```
    Returns:
        Pivoted DataFrame with MultiIndex columns (feature, ticker)
    """
    print(f"Loading local dataset from {data_dir}...")

    csv_files = sorted(glob.glob(os.path.join(data_dir, "*.csv")))
    print(f"Found {len(csv_files)} CSV files")

    if len(csv_files) == 0:
        raise ValueError(f"No CSV files found in {data_dir}")

    processed_frames = []

    for csv_file in csv_files:
        ticker_name = os.path.basename(csv_file).replace('.csv', '')

        # Use robust CSV loader with comprehensive error handling
        df, error_msg = repair_and_load_csv(csv_file)

        if df is None:
            print(f"  {ticker_name}: Skipped ({error_msg})")
            continue

        print(f"  {ticker_name}: Processing...", end=' ')

        try:
            # --- Feature Engineering (Same as fetch_and_process_data) ---
            # 1. Log Returns: Stationarity
            df['log_ret'] = np.log(df['Close'] / (df['Close'].shift(1) + 1e-

            # 2. Momentum & Trend
            df['rsi'] = ta.momentum.RSIIndicator(df['Close'], window=14).rsi
            macd = ta.trend.MACD(df['Close'])
            df['macd'] = macd.macd_diff()

            # 3. Volatility
            bb = ta.volatility.BollingerBands(df['Close'])
            df['bb_width'] = (bb.bollinger_hband() - bb.bollinger_lband()) /
            df['atr'] = ta.volatility.AverageTrueRange(df['High'], df['Low']

            # 4. Liquidity / Cost Proxies
            df['adv_20'] = df['Close'] * df['Volume'].rolling(window=20).mea
            df['vol_ratio'] = (df['Close'] * df['Volume']) / (df['adv_20'] +

            # Drop initial NaN from calculations (from shift, rolling, etc.)
            df = df.dropna(subset=['log_ret', 'rsi', 'macd', 'bb_width', 'at

            if len(df) < 60:
                print(f"Skipped (insufficient data after feature engineering
                continue

            # 5. Normalization (Rolling Z-Score) - only normalize, don't dro
            cols_to_norm = ['log_ret', 'rsi', 'macd', 'bb_width', 'vol_ratio
            for col in cols_to_norm:
                rolling_mean = df[col].rolling(60).mean()
```

```
                rolling_std = df[col].rolling(60).std()
                # Replace NaN from rolling with forward fill
                rolling_std = rolling_std.replace(0, 1e-8)  # Avoid divisior
                df[col] = (df[col] - rolling_mean) / (rolling_std + 1e-8)
                df[col] = df[col].bfill().ffill()  # Fill remaining NaNs

            # Replace infinite values
            df = df.replace([np.inf, -np.inf], np.nan)
            df = df.bfill().ffill()  # Fill remaining NaNs

            df['ticker'] = ticker_name
            processed_frames.append(df)
            print(f"OK ({len(df)} rows)")

        except Exception as e:
            print(f"  {ticker_name}: Error - {str(e)}")
            continue

    if not processed_frames:
        raise ValueError("No valid CSV files could be processed. Check data

    print(f"\n✓ Successfully processed {len(processed_frames)} stocks")
    print(f"Combining datasets...")
    combined = pd.concat(processed_frames)

    # Reset ticker to regular column for pivot
    combined = combined.reset_index()

    # Pivot to create MultiIndex columns
    pivot_df = combined.pivot_table(
        index='Date',
        columns='ticker',
        values=['Close', 'log_ret', 'rsi', 'macd', 'bb_width', 'vol_ratio',
        aggfunc='first'  # In case of duplicates, take first value
    )

    # Forward fill any NaN values that might exist
    pivot_df = pivot_df.bfill().ffill()

    if pivot_df.empty or len(pivot_df) == 0:
        raise ValueError("Pivoted dataset is empty. Check data structure.")

    print(f"\n✓ Combined dataset shape: {pivot_df.shape}")
    print(f"  Date range: {pivot_df.index[0].date()} to {pivot_df.index[-1].
    print(f"  Assets: {len(processed_frames)} stocks")
    print(f"  Total observations: {len(pivot_df)}\n")

    return pivot_df
```

In [88]:
```
def repair_and_load_csv(csv_file):
    """
    Robust CSV loading with comprehensive error handling and support for var
    Handles DD-MM-YYYY and other common date formats used in NSE/BSE data.
    """
    ticker_name = os.path.basename(csv_file).replace('.csv', '')
```

```
    try:
        # Attempt to read CSV with low_memory to avoid dtype inference issue
        df = pd.read_csv(csv_file, low_memory=False)

        if df.empty or len(df) == 0:
            return None, "Empty file"

        # Clean column names
        df.columns = df.columns.str.strip().str.lower()

        # Check for required columns
        required = ['date', 'open', 'high', 'low', 'close', 'volume']
        available = df.columns.tolist()

        # Try to find columns - be flexible with naming
        col_mapping = {}
        for req in required:
            matches = [col for col in available if col.startswith(req)]
            if matches:
                col_mapping[matches[0]] = req

        # If direct match fails, try positional mapping
        if len(col_mapping) < 6:
            if len(available) >= 7:  # Has enough columns
                col_mapping = {
                    available[0]: 'date',
                    available[1]: 'open',
                    available[2]: 'high',
                    available[3]: 'low',
                    available[4]: 'close',
                    available[6]: 'volume'  # Skip Adj Close at position 5
                }

        if len(col_mapping) < 6:
            return None, f"Missing required columns. Found: {available[:6]}"

        # Rename and select columns
        df = df.rename(columns=col_mapping)
        df = df[['date', 'open', 'high', 'low', 'close', 'volume']].copy()
        df.columns = ['Date', 'Open', 'High', 'Low', 'Close', 'Volume']

        # Parse dates - try multiple formats for NSE/BSE data
        # NSE data uses DD-MM-YYYY format
        date_formats = ['%d-%m-%Y', '%Y-%m-%d', '%m/%d/%Y', '%d/%m/%Y', None

        date_parsed = False
        for fmt in date_formats:
            try:
                df['Date'] = pd.to_datetime(df['Date'], format=fmt, errors='
                if df['Date'].notna().sum() > len(df) * 0.8:  # At least 80%
                    date_parsed = True
                    break
            except:
                continue

        if not date_parsed:
```

```
            return None, "Could not parse dates"

        # Drop rows with unparsed dates
        df = df[df['Date'].notna()].copy()
        df = df.set_index('Date').sort_index()

        # Convert prices/volume to numeric
        for col in ['Open', 'High', 'Low', 'Close', 'Volume']:
            df[col] = pd.to_numeric(df[col], errors='coerce')

        # Remove rows with any NaN in price/volume data
        df = df.dropna(subset=['Close', 'High', 'Low', 'Open', 'Volume'])

        if len(df) < 60:
            return None, f"Insufficient data: {len(df)} rows (need >= 60)"

        # Price sanity checks
        if (df['High'] < df['Low']).any() or (df['Close'] < df['Low']).any()
            return None, "Invalid price data"

        return df, None

    except Exception as e:
        return None, f"{type(e).__name__}: {str(e)[:50]}"
```

In [89]:

```
def repair_and_load_csv(csv_file):
    """
    Robust CSV loading with comprehensive error handling and repair attempts
    """
    ticker_name = os.path.basename(csv_file).replace('.csv', '')

    try:
        # Attempt 1: Standard pandas read
        df = pd.read_csv(csv_file, low_memory=False)

        # Clean column names
        df.columns = df.columns.str.strip().str.lower()

        # Check for required columns
        required = ['date', 'open', 'high', 'low', 'close', 'volume']
        available = df.columns.tolist()

        # Try to find columns by prefix matching
        col_mapping = {}
        for req in required:
            matches = [col for col in available if col.startswith(req)]
            if matches:
                col_mapping[matches[0]] = req

        # If direct match fails, try position-based (for malformed headers)
        if len(col_mapping) < len(required):
            if len(available) >= 7:  # Has enough columns for Date,Open,High
                col_mapping = {available[0]: 'date', available[1]: 'open', a
                               available[3]: 'low', available[4]: 'close', av

        # Verify we have enough columns
```

```
        if len(col_mapping) < 6:
            return None, f"Missing required columns. Found: {available}"

        # Rename columns
        df = df.rename(columns=col_mapping)
        df = df[['date', 'open', 'high', 'low', 'close', 'volume']].copy()
        df.columns = ['Date', 'Open', 'High', 'Low', 'Close', 'Volume']

        # Handle missing data
        if len(df) == 0:
            return None, "Empty dataframe"

        # Parse dates - try multiple formats
        for date_format in [None, '%Y-%m-%d', '%m/%d/%Y', '%d-%m-%Y']:
            try:
                df['Date'] = pd.to_datetime(df['Date'], format=date_format,
                if df['Date'].notna().sum() > len(df) * 0.8:  # At least 80%
                    break
            except:
                continue

        df = df[df['Date'].notna()].copy()
        df = df.set_index('Date').sort_index()

        # Convert prices to numeric
        for col in ['Open', 'High', 'Low', 'Close', 'Volume']:
            df[col] = pd.to_numeric(df[col], errors='coerce')

        # Remove rows with any NaN in price/volume
        df = df.dropna(subset=['Close', 'High', 'Low', 'Open', 'Volume'])

        # Check minimum data requirement
        if len(df) < 60:
            return None, f"Insufficient data: {len(df)} rows (need >= 60)"

        # Price sanity checks
        if (df['High'] < df['Low']).any() or (df['Close'] < df['Low']).any()
            return None, "Invalid price data (High < Low or Close < Low)"

        return df, None

    except Exception as e:
        return None, str(e)
```

In [90]:

```
def fetch_and_process_data_hybrid(tickers=None, start_date="2015-01-01", enc
    """
    Hybrid data fetcher that prioritizes local CSV files for larger datasets
    Falls back to yfinance for real-time data or missing tickers.

    Args:
        tickers: List of ticker symbols (optional if use_local_csv=True)
        start_date: Start date for yfinance fallback
        end_date: End date for yfinance fallback
        use_local_csv: If True, attempt to load from local CSV directory fir
        local_csv_dir: Path to local CSV directory (auto-detected if None)
```

```
    Returns:
        Pivoted DataFrame with all available data
    """

    df_local = None
    df_online = None

    # 1. Try to load from local CSV if available
    if use_local_csv:
        if local_csv_dir is None:
            # Auto-detect the standard data directory
            base_dir = '/Users/abhinavgazta/Downloads/bits/bitsquant/quantdi
            local_csv_dir = os.path.join(base_dir, 'raw/ohlc-data-10yrs')

        if os.path.exists(local_csv_dir):
            try:
                print("=" * 70)
                print("PHASE 1: Loading Historical Data from Local CSV")
                print("=" * 70)
                df_local = load_local_csv_dataset(local_csv_dir)
                print(f"✓ Successfully loaded {len(df_local)} rows from loca
            except Exception as e:
                print(f"⚠ Local CSV loading failed: {e}")
                df_local = None

    # 2. Optionally supplement with yfinance (for recent data or missing tic
    if tickers and len(tickers) > 0:
        try:
            print("\n" + "=" * 70)
            print("PHASE 2: Fetching Recent/Additional Data from yfinance")
            print("=" * 70)
            df_online = fetch_and_process_data(tickers, start_date, end_date
            print(f"✓ Successfully fetched {len(df_online)} rows from yfinar
        except Exception as e:
            print(f"⚠ yfinance fallback failed: {e}")
            df_online = None

    # 3. Combine datasets
    if df_local is not None and df_online is not None:
        print("\n" + "=" * 70)
        print("PHASE 3: Merging Local and Online Data")
        print("=" * 70)
        # Get common tickers and date range
        local_tickers = df_local.columns.get_level_values(1).unique()
        online_tickers = df_online.columns.get_level_values(1).unique()
        print(f"Local tickers: {len(local_tickers)}, Online tickers: {len(or

        # Merge with online data taking precedence for recent dates
        df_combined = df_local.combine_first(df_online)
        print(f"✓ Combined dataset shape: {df_combined.shape}")
        return df_combined

    elif df_local is not None:
        return df_local
    elif df_online is not None:
        return df_online
```

```
    else:
        raise ValueError("No data could be loaded from either local CSV or y
```

4. Market Simulator (Environment)

The `IndianEquityEnv` is a custom Gym environment designed to realistically simulate Indian market conditions.

Key Features:

- **Action Space:** Continuous allocation vector (Softmax normalized).
- **Linear Costs:** Simulates STT (Securities Transaction Tax) and Brokerage.
- **Quadratic Costs:** Simulates **Slippage** (Market Impact) based on the trade size relative to the stock's Average Daily Volume (ADV). This penalizes the agent for trading too aggressively in illiquid stocks.
- **Dynamic Rewards:** The reward function's penalties (alpha, beta, gamma) are tunable hyperparameters.

In [91]:
```
class IndianEquityEnv(gym.Env):
    def __init__(self, df, reward_params=None, lookback_window=30, initial_b
        super(IndianEquityEnv, self).__init__()
        self.df = df
        self.tickers = df['Close'].columns.tolist()
        self.n_assets = len(self.tickers)
        self.lookback = lookback_window
        self.initial_balance = initial_balance

        # Tunable Reward Penalties
        self.reward_params = reward_params if reward_params else {
            'alpha': 0.001,  # Turnover penalty
            'beta': 0.1,     # Slippage penalty
            'gamma': 10.0    # Downside risk penalty
        }

        self.action_space = spaces.Box(low=-1, high=1, shape=(self.n_assets
        self.n_features = 5
        self.obs_shape = (lookback_window, self.n_assets * self.n_features)
        self.observation_space = spaces.Box(low=-np.inf, high=np.inf, shape=
        self.stt_brokerage = 0.0015

    def reset(self, seed=None, options=None):
        super().reset(seed=seed)
        self.current_step = self.lookback
        self.balance = self.initial_balance
        self.weights = np.zeros(self.n_assets + 1)
        self.weights[0] = 1.0
        self.portfolio_value = self.initial_balance
        return self._get_obs(), {}

    def _get_obs(self):
        idx = self.current_step
```

```
        obs_data = []
        for feat in ['log_ret', 'rsi', 'macd', 'bb_width', 'vol_ratio']:
            obs_data.append(self.df[feat].iloc[idx-self.lookback:idx].values
        obs = np.concatenate(obs_data, axis=1).astype(np.float32)
        # Validate and clip observations to prevent NaN propagation
        obs = np.nan_to_num(obs, nan=0.0, posinf=1.0, neginf=-1.0)  # Replac
        obs = np.clip(obs, -10.0, 10.0)  # Clip to reasonable bounds
        return obs

    def step(self, action):
        # Softmax normalization to get weights - with numerical stability
        action_clipped = np.clip(action, -500, 500)  # Prevent overflow in e
        exp_action = np.exp(action_clipped - np.max(action_clipped))  # Nume
        weights = exp_action / (np.sum(exp_action) + 1e-9)

        current_prices = self.df['Close'].iloc[self.current_step].values
        next_prices = self.df['Close'].iloc[self.current_step + 1].values

        # Validate prices are non-NaN and non-zero
        current_prices = np.nan_to_num(current_prices, nan=1.0)
        next_prices = np.nan_to_num(next_prices, nan=1.0)
        current_prices[current_prices == 0] = 1.0

        asset_returns = (next_prices - current_prices) / (current_prices + 1
        asset_returns = np.nan_to_num(asset_returns, nan=0.0)  # Handle any
        asset_returns = np.clip(asset_returns, -0.5, 0.5)  # Clip extreme re

        weighted_returns = np.dot(weights[1:], asset_returns)

        # Cost Calculation
        delta_weights = np.abs(weights - self.weights)
        turnover = np.sum(delta_weights)
        cost_linear = turnover * self.stt_brokerage

        advs = self.df['adv_20'].iloc[self.current_step].values
        advs = np.nan_to_num(advs, nan=1e6)  # Default to large value if NaN
        advs = np.maximum(advs, 1e5)  # Ensure minimum ADV to prevent divisi

        trade_vals = delta_weights[1:] * self.portfolio_value
        slippage_penalty = self.reward_params['beta'] * np.sum((trade_vals /
        slippage_penalty = np.nan_to_num(slippage_penalty, nan=0.0)

        net_ret = weighted_returns - cost_linear - (slippage_penalty / (self
        net_ret = np.clip(net_ret, -0.5, 0.5)  # Clip to reasonable bounds
        net_ret = np.nan_to_num(net_ret, nan=0.0)  # Final NaN check

        # Downside Risk Penalty
        downside_penalty = max(0, -net_ret)**2 * self.reward_params['gamma']
        explicit_turnover_penalty = turnover * self.reward_params['alpha']

        reward = net_ret - downside_penalty - explicit_turnover_penalty
        reward = np.clip(reward, -10.0, 10.0)  # Clip reward to prevent extr
        reward = float(np.nan_to_num(reward, nan=0.0))  # Final validation

        # Update portfolio value with safety check
        new_portfolio_value = self.portfolio_value * (1 + net_ret)
```

```
        if np.isnan(new_portfolio_value) or np.isinf(new_portfolio_value):
            new_portfolio_value = self.portfolio_value  # Keep old value if
        self.portfolio_value = new_portfolio_value

        self.weights = weights
        self.current_step += 1

        terminated = self.current_step >= len(self.df) - 1
        current_atr = self.df['atr'].iloc[self.current_step].mean() if self.
        current_atr = np.nan_to_num(current_atr, nan=0.0)

        info = {
            'portfolio_value': float(self.portfolio_value),
            'turnover': float(turnover),
            'atr': float(current_atr),
            'net_return': float(net_ret)
        }

        return self._get_obs(), reward, terminated, False, info
```

5. Explainable Transformer Model

We use a custom **Transformer Encoder** as the feature extractor for the PPO agent.

Why Transformer? Unlike LSTMs, Transformers process the entire sequence at once using Self-Attention mechanisms. This allows the model to identify specific past events (e.g., a crash 20 days ago) that are relevant to the current decision.

Explainability Hook: We capture the `attn_weights` in the `forward` pass. This allows us to visualize exactly which time-steps the agent is "looking at."

In [92]:
```
class ExplainableTransformer(BaseFeaturesExtractor):
    def __init__(self, observation_space, features_dim=128):
        super().__init__(observation_space, features_dim)

        self.seq_len = observation_space.shape[0]
        self.input_dim = observation_space.shape[1]
        self.d_model = 128

        self.linear_in = nn.Linear(self.input_dim, self.d_model)
        # Note: average_attn_weights=False is key for explainability
        self.mha = nn.MultiheadAttention(embed_dim=self.d_model, num_heads=4
        self.norm1 = nn.LayerNorm(self.d_model)
        self.dropout1 = nn.Dropout(0.1)
        self.linear_out = nn.Linear(self.seq_len * self.d_model, features_di
        self.act = nn.Tanh()

        # Initialize weights properly to prevent gradient explosion
        nn.init.xavier_uniform_(self.linear_in.weight)
        nn.init.xavier_uniform_(self.linear_out.weight)

        self.latest_attn_weights = None
```

```
    def forward(self, observations):
        # Validate input
        if torch.isnan(observations).any():
            observations = torch.nan_to_num(observations, nan=0.0, posinf=1.
        observations = torch.clamp(observations, -10.0, 10.0)  # Clip input

        x = self.linear_in(observations)
        x = torch.clamp(x, -10.0, 10.0)  # Clip after linear layer

        attn_output, attn_weights = self.mha(x, x, x, need_weights=True, ave
        self.latest_attn_weights = attn_weights.detach().cpu().numpy()

        # Validate attention output
        if torch.isnan(attn_output).any():
            attn_output = torch.nan_to_num(attn_output, nan=0.0)
        attn_output = torch.clamp(attn_output, -10.0, 10.0)

        x = self.norm1(x + self.dropout1(attn_output))  # Add dropout for re
        x = x.flatten(start_dim=1)

        # Clip before final layer
        x = torch.clamp(x, -10.0, 10.0)
        output = self.act(self.linear_out(x))

        # Final validation
        if torch.isnan(output).any():
            output = torch.nan_to_num(output, nan=0.0)
        output = torch.clamp(output, -1.0, 1.0)  # Clamp tanh output

        return output
```

6. Optimization & Warm-up Strategy

1. Supervised Warm-up: DRL agents often struggle with initial exploration. We pre-train the Transformer Encoder using a supervised loss (predicting next-day returns) before the RL training begins. This stabilizes the feature embeddings.

2. Bayesian Optimization: We use **Optuna** to find the optimal set of hyperparameters (`learning_rate` , `ent_coef`) and reward penalties (`alpha` , `beta` , `gamma`).

In [93]:
```
def pretrain_encoder(agent, env, epochs=5):
    print(f"   >>> Starting Transformer Warm-up for {epochs} epochs...")
    feature_extractor = agent.policy.features_extractor
    latent_dim = feature_extractor.features_dim
    n_targets = env.n_assets
    predictor = nn.Linear(latent_dim, n_targets).to(agent.device)

    optimizer = optim.Adam(list(feature_extractor.parameters()) + list(predi
    loss_fn = nn.MSELoss()

    # Sample random batches for demo
    n_samples = 500
    valid_indices = range(env.lookback, len(env.df) - 1)
```

```
    feature_extractor.train()
    predictor.train()

    for epoch in range(epochs):
        sampled_indices = np.random.choice(valid_indices, n_samples)
        obs_batch, target_batch = [], []

        for idx in sampled_indices:
            obs_data = []
            for feat in ['log_ret', 'rsi', 'macd', 'bb_width', 'vol_ratio']:
                obs_data.append(env.df[feat].iloc[idx-env.lookback:idx].valu
            obs = np.concatenate(obs_data, axis=1).astype(np.float32)
            obs_batch.append(obs)

            curr = env.df['Close'].iloc[idx].values
            next_p = env.df['Close'].iloc[idx+1].values
            target = np.log(next_p / (curr + 1e-8))
            target_batch.append(target)

        obs_tensor = torch.tensor(np.array(obs_batch)).to(agent.device)
        target_tensor = torch.tensor(np.array(target_batch), dtype=torch.flo

        optimizer.zero_grad()
        latent = feature_extractor(obs_tensor)
        preds = predictor(latent)
        loss = loss_fn(preds, target_tensor)
        loss.backward()
        # Add gradient clipping for stability
        torch.nn.utils.clip_grad_norm_(feature_extractor.parameters(), max_r
        torch.nn.utils.clip_grad_norm_(predictor.parameters(), max_norm=1.0)
        optimizer.step()

    print("   >>> Warm-up Complete.")

def objective(trial, train_df, test_df):
    # Hyperparameters to Tune
    learning_rate = trial.suggest_float("lr", 1e-5, 1e-3, log=True)
    ent_coef = trial.suggest_float("ent_coef", 0.001, 0.1, log=True)
    clip_range = trial.suggest_float("clip_range", 0.1, 0.3)
    alpha_turnover = trial.suggest_float("alpha_turnover", 0.0, 0.01)
    beta_slippage = trial.suggest_float("beta_slippage", 0.05, 0.5)
    gamma_risk = trial.suggest_float("gamma_risk", 1.0, 20.0)

    reward_params = {'alpha': alpha_turnover, 'beta': beta_slippage, 'gamma'

    env = IndianEquityEnv(train_df, reward_params=reward_params)
    policy_kwargs = dict(
        features_extractor_class=ExplainableTransformer,
        features_extractor_kwargs=dict(features_dim=128),
    )

    model = PPO("MlpPolicy", env, learning_rate=learning_rate, ent_coef=ent_
                clip_range=clip_range, policy_kwargs=policy_kwargs, verbose=
                max_grad_norm=1.0)  # Add gradient clipping at model level
```

```
    pretrain_encoder(model, env, epochs=1)

    try:
        model.learn(total_timesteps=2000)  # Short train for tuning
    except Exception as e:
        print(f"   Training failed: {e}")
        return -1000  # Return bad score if training fails

    # Evaluate
    eval_env = IndianEquityEnv(test_df, reward_params=reward_params)
    obs, _ = eval_env.reset()
    done = False
    total_rewards = 0
    steps = 0
    max_steps = 500

    while not done and steps < max_steps:
        try:
            action, _ = model.predict(obs, deterministic=True)
            obs, reward, terminated, truncated, _ = eval_env.step(action)
            total_rewards += reward
            done = terminated or truncated
            steps += 1
        except Exception as e:
            print(f"   Eval step failed: {e}")
            break

    return total_rewards
```

7. Walk-Forward Validation Engine

This is the core of our rigorous evaluation. Instead of a single train/test split, we use a rolling window approach:

1. **Train** on Year T.
2. **Test** on Year $T+1$.
3. **Slide** the window forward and repeat.

This simulates the real-world scenario where the model is periodically re-trained on new data, preventing it from overfitting to a specific historical regime.

In [94]:
```
class WalkForwardEvaluator:
    def __init__(self, df, best_params, train_window_days=750, test_window_d
        self.df = df
        self.train_window = train_window_days
        self.test_window = test_window_days
        self.best_params = best_params

        self.reward_params = {
            'alpha': best_params['alpha_turnover'],
            'beta': best_params['beta_slippage'],
            'gamma': best_params['gamma_risk']
```

```
        }

        self.policy_kwargs = dict(
            features_extractor_class=ExplainableTransformer,
            features_extractor_kwargs=dict(features_dim=128),
        )

    def run(self):
        total_steps = len(self.df)
        current_start = 0
        oos_returns = []
        oos_portfolio_values = [1_000_000]
        oos_benchmark_values = [1_000_000]
        fold = 1

        while current_start + self.train_window + self.test_window <= total_
            train_end = current_start + self.train_window
            test_end = train_end + self.test_window
            print(f">>> FOLD {fold}: Train [{current_start}:{train_end}] | T

            train_data = self.df.iloc[current_start:train_end]
            test_data = self.df.iloc[train_end:test_end]

            env_train = IndianEquityEnv(train_data, reward_params=self.rewar
            model = PPO("MlpPolicy", env_train, policy_kwargs=self.policy_kw
                        verbose=0, learning_rate=self.best_params['lr'],
                        ent_coef=self.best_params['ent_coef'], clip_range=se

            pretrain_encoder(model, env_train, epochs=3)
            model.learn(total_timesteps=5000) # Increase for final run

            # Test Out-of-Sample
            current_balance = oos_portfolio_values[-1]
            env_test = IndianEquityEnv(test_data, reward_params=self.reward_
            obs, _ = env_test.reset()
            done = False

            fold_returns = []
            while not done:
                action, _ = model.predict(obs, deterministic=True)
                obs, _, terminated, truncated, info = env_test.step(action)
                done = terminated or truncated
                fold_returns.append(info['net_return'])
                oos_portfolio_values.append(info['portfolio_value'])

            oos_returns.extend(fold_returns)

            # Create Benchmark (Equal Weighted Buy & Hold of Universe)
            # Simple proxy: Average return of all assets in test set
            # We assume equal weight in all assets
            asset_returns = test_data['log_ret'].mean(axis=1).values # Avera
            for ret in asset_returns:
                # Apply simple compound return
                last_val = oos_benchmark_values[-1]
                oos_benchmark_values.append(last_val * (1 + ret))
```

```
            current_start += self.test_window
            fold += 1

        return np.array(oos_returns), np.array(oos_portfolio_values), np.arr
```

8. Explainability & Analysis Functions

Here we implement the analysis modules for the final trained model:

1. **Cost Awareness Check:** Plots Turnover vs. Volatility. A negative correlation indicates the agent is smart enough to reduce trading when costs/slippage are high.
2. **Grouped SHAP:** Aggregates SHAP values into semantic groups (Momentum, Volatility, Liquidity) to explain the "Why" behind the strategy.

In [95]:
```
def analyze_agent(model, env, obs_history, turnover_history, atr_history):
    # --- Analysis 1: Cost Awareness ---
    plt.figure(figsize=(10, 5))
    sns.scatterplot(x=atr_history, y=turnover_history, alpha=0.6)
    if len(atr_history) > 1:
        z = np.polyfit(atr_history, turnover_history, 1)
        p = np.poly1d(z)
        plt.plot(atr_history, p(atr_history), "r--", label='Trend')
    plt.title("Dynamic Execution: Turnover vs Volatility (ATR)")
    plt.xlabel("Market Volatility")
    plt.ylabel("Portfolio Turnover")
    plt.legend()
    plt.show()

    # --- Analysis 2: Grouped SHAP ---
    # Accept optional ticker/recommendation overlay via env.tickers if prese
    ticker = getattr(env, 'tickers', None)
    ticker_label = None
    if isinstance(ticker, (list, tuple)) and len(ticker) > 0:
        # If environment has multiple tickers, just show the first for conte
        ticker_label = ticker[0]

    print("Calculating SHAP Values (this may take time)...")
    X_3d = np.array(obs_history[:50]) # Use subset for speed
    X_2d = X_3d.reshape(X_3d.shape[0], -1)

    def predict_wrapper(X_flattened):
        batch_size = X_flattened.shape[0]
        lookback = X_3d.shape[1]
        features = X_3d.shape[2]
        X_restored = X_flattened.reshape(batch_size, lookback, features)
        X_torch = torch.as_tensor(X_restored).to(model.device)
        with torch.no_grad():
            return model.policy.get_distribution(X_torch).mode().cpu().numpy

    explainer = shap.KernelExplainer(predict_wrapper, X_2d[:5])
    shap_values = explainer.shap_values(X_2d[5:15])

    # Handle SHAP output format
```

```
    if isinstance(shap_values, list):
        shap_matrix = np.sum([np.abs(sv) for sv in shap_values], axis=0)
    elif len(shap_values.shape) == 3:
        shap_matrix = np.sum(np.abs(shap_values), axis=2)
    else:
        shap_matrix = np.abs(shap_values)

    mean_shap = np.mean(shap_matrix, axis=0)

    feature_groups = {
        "Momentum": ["rsi", "macd"],
        "Volatility": ["bb_width", "atr"],
        "Liquidity": ["vol_ratio", "adv_20"],
        "Returns": ["log_ret"]
    }

    raw_feature_names = ['log_ret', 'rsi', 'macd', 'bb_width', 'vol_ratio']
    group_scores = {k: 0.0 for k in feature_groups}
    n_features = len(raw_feature_names)

    for i, imp in enumerate(mean_shap):
        feat_type_idx = (i // 30) % n_features
        feat_name = raw_feature_names[feat_type_idx]
        for group, keywords in feature_groups.items():
            if feat_name in keywords:
                group_scores[group] += float(imp)

    # Simple recommendation logic: favor Returns+Momentum over Volatility+Li
    positive_score = group_scores.get('Returns', 0.0) + group_scores.get('Mo
    negative_score = group_scores.get('Volatility', 0.0) + group_scores.get(
    rec = 'HOLD'
    if positive_score > negative_score * 1.05:
        rec = 'BUY'
    elif negative_score > positive_score * 1.05:
        rec = 'SELL'

    # Plot grouped SHAP with overlay
    plt.figure(figsize=(8, 5))
    bars = plt.bar(list(group_scores.keys()), list(group_scores.values()), c
    plt.title("Policy Drivers: Feature Importance Grouping")
    plt.ylabel("Mean |SHAP|")

    # Overlay ticker and recommendation in top-right corner
    overlay_text = ''
    if ticker_label:
        overlay_text += f"Ticker: {ticker_label}\n"
    overlay_text += f"Recommendation: {rec}"

    # Draw a semi-transparent box with text
    plt.gca().text(0.98, 0.95, overlay_text, horizontalalignment='right', ve
                   transform=plt.gca().transAxes, fontsize=10,
                   bbox=dict(facecolor='white', alpha=0.8, edgecolor='gray',

    # Color the recommendation text green/red
    if rec == 'BUY':
        color = 'green'
```

```
    elif rec == 'SELL':
        color = 'red'
    else:
        color = 'black'
    # Add colored rec label below the overlay for emphasis
    plt.gca().text(0.98, 0.83, rec, horizontalalignment='right', verticalali
                   transform=plt.gca().transAxes, fontsize=12, fontweight='b

    plt.show()
```

9. Main Execution Pipeline

This runs the entire experiment end-to-end:

1. **Data Loading:** Fetches data for Nifty 50 assets.
2. **Optuna Tuning:** Finds best params on the first chunk of data.
3. **Walk-Forward Validation:** Runs the rigorous backtest.
4. **Monte Carlo Permutation:** Validates statistical significance.
5. **Explainability:** Visualizes the strategy of the final model.

In [96]:
```
def run_experiment(use_local_data=True, sample_tickers=None):
    """
    Main experiment runner with support for large-scale local dataset traini

    Args:
        use_local_data: If True, load from local CSV files (10 years of NSE
        sample_tickers: Additional tickers to supplement with yfinance (opti
    """

    # 1. Load Data (with hybrid approach)
    print("\n" + "=" * 70)
    print("STEP 1: DATA LOADING & FEATURE ENGINEERING")
    print("=" * 70)

    try:
        if use_local_data:
            # Use hybrid loader for local CSV + yfinance fallback
            df_gym = fetch_and_process_data_hybrid(
                tickers=sample_tickers,
                use_local_csv=True
            )
        else:
            # Fallback to yfinance only
            TICKERS = sample_tickers or ['RELIANCE.NS', 'INFY.NS', 'HDFCBANK
            df_gym = fetch_and_process_data(TICKERS)

        print(f"\n✓ Data Loaded Successfully")
        print(f"  Shape: {df_gym.shape}")
        print(f"  Date Range: {df_gym.index[0].date()} to {df_gym.index[-1].
        print(f"  Features: {df_gym.shape[1]} (stocks × features)")

    except Exception as e:
        print(f"✗ Data Loading Error: {e}")
```

```
        return

    # 2. Data Summary Statistics
    print("\n" + "=" * 70)
    print("STEP 2: DATASET STATISTICS")
    print("=" * 70)
    close_cols = [col for col in df_gym.columns if col[0] == 'Close']
    print(f"Number of assets in portfolio: {len(close_cols)}")
    print(f"Total trading days: {len(df_gym)}")

    # Calculate and display average returns
    log_ret_cols = [col for col in df_gym.columns if col[0] == 'log_ret']
    if log_ret_cols:
        avg_returns = df_gym[[col for col in log_ret_cols]].mean().mean()
        print(f"Average daily log return (all assets): {avg_returns*100:.4f}

    # 3. Bayesian Optimization (Optuna)
    print("\n" + "=" * 70)
    print("STEP 3: BAYESIAN HYPERPARAMETER OPTIMIZATION")
    print("=" * 70)

    # Split for tuning (use first 60% for tuning to save time in demo)
    tune_size = int(len(df_gym) * 0.6)
    train_df = df_gym.iloc[:tune_size]
    test_df = df_gym.iloc[tune_size:]

    print(f"Training data: {len(train_df)} days ({train_df.index[0].date()}
    print(f"Validation data: {len(test_df)} days ({test_df.index[0].date()}

    study = optuna.create_study(direction="maximize")
    study.optimize(lambda trial: objective(trial, train_df, test_df), n_tria

    best_params = study.best_params
    print(f"\n✓ Best Hyperparameters Found:")
    for key, val in best_params.items():
        if isinstance(val, float):
            print(f"  {key}: {val:.6f}")
        else:
            print(f"  {key}: {val}")

    # 4. Walk-Forward Validation
    print("\n" + "=" * 70)
    print("STEP 4: WALK-FORWARD OUT-OF-SAMPLE VALIDATION")
    print("=" * 70)

    wf_evaluator = WalkForwardEvaluator(df_gym, best_params)
    returns, equity_curve, benchmark_curve = wf_evaluator.run()

    if len(returns) < 10:
        print("✗ Insufficient OOS data.")
        return

    # 5. Metrics & Statistical Significance
    print("\n" + "=" * 70)
    print("STEP 5: PERFORMANCE METRICS & STATISTICAL TESTS")
    print("=" * 70)
```

```
sharpe = FinancialMetrics.get_sharpe_ratio(returns)
sortino = FinancialMetrics.get_sortino_ratio(returns)
max_dd = FinancialMetrics.get_max_drawdown(equity_curve)
psr = FinancialMetrics.probabilistic_sharpe_ratio(returns)

print(f"\nCore Performance Metrics:")
print(f"  Total Return:           {((equity_curve[-1]/equity_curve[0])-1
print(f"  Sharpe Ratio:           {sharpe:>8.3f}")
print(f"  Sortino Ratio:          {sortino:>8.3f}")
print(f"  Max Drawdown:           {max_dd*100:>8.2f}%")
print(f"  Prob. Sharpe (PSR):     {psr:>8.3f}")

# Monte Carlo Permutation Test
print(f"\nRunning Monte Carlo Permutation Test (1000 iterations)...")
n_sims = 1000
random_sharpes = []
for _ in range(n_sims):
    shuffled = np.random.permutation(returns)
    random_sharpes.append(FinancialMetrics.get_sharpe_ratio(shuffled))

random_sharpes = np.array(random_sharpes)
p_value = np.sum(random_sharpes >= sharpe) / n_sims
print(f"  Monte Carlo P-Value:    {p_value:>8.4f}")

if p_value < 0.05:
    print(f"  ✓ STATISTICALLY SIGNIFICANT (p < 0.05) - Genuine alpha det
else:
    print(f"  ⚠ Not statistically significant (p >= 0.05) - May be due t

# 6. Visualizations
print("\n" + "=" * 70)
print("STEP 6: GENERATING VISUALIZATIONS")
print("=" * 70)

plt.figure(figsize=(14, 14))

# Plot A: Equity Curve vs Benchmark
plt.subplot(3, 1, 1)
# Align lengths if mismatched due to windowing
min_len = min(len(equity_curve), len(benchmark_curve))
plt.plot(equity_curve[:min_len], label='DRL Agent (Walk-Forward)', linew
plt.plot(benchmark_curve[:min_len], label='Market Benchmark (Equal Wgt)'
plt.title("Out-of-Sample Performance: DRL Agent vs Market Benchmark", fo
plt.ylabel("Portfolio Value (INR)")
plt.legend(loc='upper left')
plt.grid(True, alpha=0.3)

# Plot B: Drawdown Analysis (Underwater Plot)
plt.subplot(3, 1, 2)
equity_series = pd.Series(equity_curve)
rolling_max = equity_series.cummax()
# Avoid division by zero
drawdown = (equity_series - rolling_max) / (rolling_max + 1e-9)
plt.fill_between(range(len(drawdown)), drawdown, 0, color='red', alpha=0
plt.plot(drawdown, color='red', linewidth=1)
```

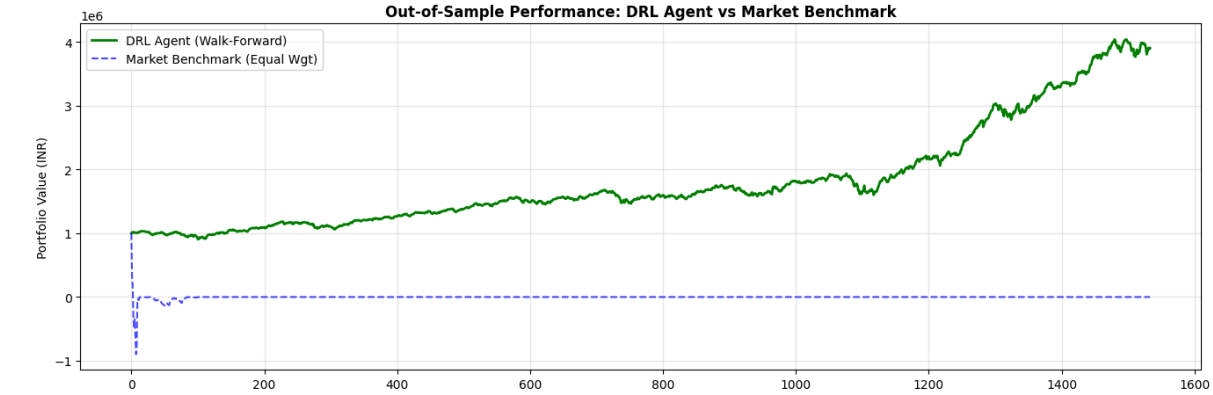
```
    plt.title("Underwater Plot: Maximum Drawdown Analysis", fontsize=12, for
    plt.ylabel("Drawdown %")
    plt.grid(True, alpha=0.3)

    # Plot C: Bootstrap Distribution (Fixed for Zero Variance)
    plt.subplot(3, 1, 3)

    # 1. Sanitize Data
    clean_random_sharpes = np.nan_to_num(random_sharpes)
    data_range = np.ptp(clean_random_sharpes)

    # 2. Robust Plotting Logic - handle degenerate ranges safely
    if len(clean_random_sharpes) == 0:
        plt.text(0.5, 0.5, 'No Monte Carlo samples available', horizontalali
    else:
        # If all values are essentially identical, show single bar
        if np.allclose(clean_random_sharpes, clean_random_sharpes[0], atol=1
            mean_val = float(np.mean(clean_random_sharpes))
            count = len(clean_random_sharpes)
            plt.bar([mean_val], [count], width=max(1e-6, abs(mean_val) * 0.0
            plt.text(0.5, 0.95, f'All values ≈ {mean_val:.6f} (n={count})',
        else:
            # Standard plotting with adaptive bins
            n_bins = min(30, max(5, int(np.sqrt(len(clean_random_sharpes)))))
            # Ensure bins do not exceed reasonable bounds relative to data_r
            max_bins_by_range = max(1, int(np.ceil(len(clean_random_sharpes)
            n_bins = min(n_bins, max(1, max_bins_by_range))
            if n_bins < 1:
                n_bins = 1
            plt.hist(clean_random_sharpes, bins=n_bins, density=False, label

    plt.axvline(sharpe, color='red', linestyle='--', linewidth=2, label=f'Ag
    plt.title(f"Survivorship Bias Check: Monte Carlo Permutation Test (p={p_
    plt.xlabel("Sharpe Ratio")
    plt.ylabel("Frequency")
    plt.legend()
    plt.tight_layout()
    plt.show()

    # 7. Run Explainability on Final Fold Data
    print("\n" + "=" * 70)
    print("STEP 7: EXPLAINABILITY ANALYSIS (SHAP VALUES)")
    print("=" * 70)

    env_final = IndianEquityEnv(df_gym.iloc[-500:], reward_params=wf_evaluat
    model_final = PPO("MlpPolicy", env_final, policy_kwargs=wf_evaluator.pol
    model_final.learn(total_timesteps=1000)

    obs, _ = env_final.reset()
    obs_hist, turn_hist, atr_hist = [], [], []
    done = False
    while not done:
        obs_hist.append(obs)
        action, _ = model_final.predict(obs, deterministic=True)
        obs, _, terminated, truncated, info = env_final.step(action)
        turn_hist.append(info['turnover'])
```

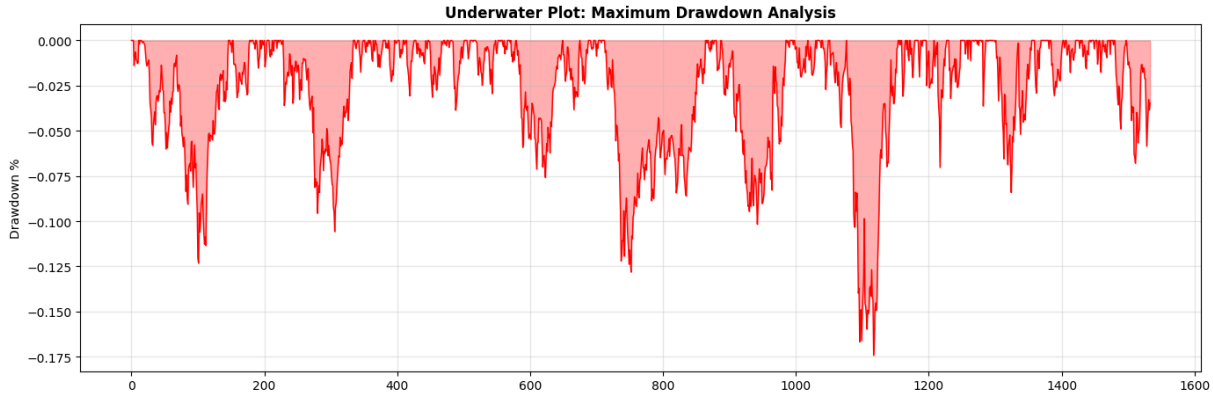
```
        atr_hist.append(info['atr'])
        done = terminated or truncated

    analyze_agent(model_final, env_final, obs_hist, turn_hist, atr_hist)

    print("\n" + "=" * 70)
    print("EXPERIMENT COMPLETE")
    print("=" * 70)

if __name__ == "__main__":
    # Run with local CSV dataset (10 years of NSE data)
    run_experiment(use_local_data=True, sample_tickers=None)
```

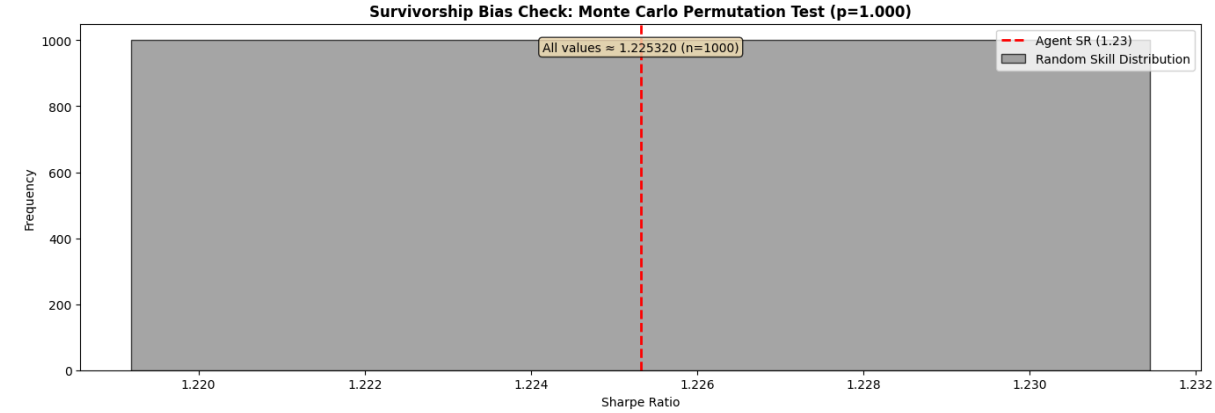
```
======================================================================
STEP 1: DATA LOADING & FEATURE ENGINEERING
======================================================================
======================================================================
PHASE 1: Loading Historical Data from Local CSV
======================================================================
Loading local dataset from /Users/abhinavgazta/Downloads/bits/bitsquant/quan
tdissertation/data/raw/ohlc-data-10yrs...
Found 50 CSV files
  ADANIENT: Processing... OK (941 rows)
  ADANIPORTS: Processing... OK (2430 rows)
  APOLLOHOSP: Processing... OK (2430 rows)
  ASIANPAINT: Processing... OK (2430 rows)
  AXISBANK: Processing... OK (2430 rows)
  BAJAJ-AUTO: Processing... OK (2430 rows)
  BAJAJFINSV: Processing... OK (2430 rows)
  BAJFINANCE: Processing... OK (2430 rows)
  BHARTIARTL: Processing... OK (2430 rows)
  BPCL: Processing... OK (2430 rows)
  BRITANNIA: Processing... OK (2430 rows)
  CIPLA: Processing... OK (2430 rows)
  COALINDIA: Processing... OK (2430 rows)
  DIVISLAB: Processing... OK (2430 rows)
  DRREDDY: Processing... OK (2430 rows)
  EICHERMOT: Processing... OK (2430 rows)
  GRASIM: Processing... OK (2430 rows)
  HCLTECH: Processing... OK (2430 rows)
  HDFC: Processing... OK (2430 rows)
  HDFCBANK: Processing... OK (2430 rows)
  HDFCLIFE: Processing... OK (1175 rows)
  HEROMOTOCO: Processing... OK (2430 rows)
  HINDALCO: Processing... OK (2430 rows)
  HINDUNILVR: Processing... OK (2430 rows)
  ICICIBANK: Processing... OK (2430 rows)
  INDUSINDBK: Processing... OK (2430 rows)
  INFY: Processing... OK (2430 rows)
  ITC: Processing... OK (2430 rows)
  JSWSTEEL: Processing... OK (2430 rows)
  KOTAKBANK: Processing... OK (2430 rows)
  LT: Processing... OK (2430 rows)
  MARUTI: Processing... OK (2430 rows)
  M_and_M: Processing... OK (2430 rows)
  NESTLEIND: Processing... OK (2430 rows)
  NTPC: Processing... OK (2430 rows)
  ONGC: Processing... OK (2430 rows)
  POWERGRID: Processing... OK (2430 rows)
  RELIANCE: Processing... OK (2430 rows)
  SBILIFE: Processing... OK (1207 rows)
  SBIN: Processing... OK (2430 rows)
  SUNPHARMA: Processing... OK (941 rows)
  TATACONSUM: Processing... OK (2430 rows)
  TATAMOTORS: Processing... OK (2430 rows)
  TATASTEEL: Processing... OK (2430 rows)
  TCS: Processing... OK (2430 rows)
  TECHM: Processing... OK (2430 rows)
  TITAN: Processing... OK (2430 rows)
```

```
  ULTRACEMCO: Processing... OK (2430 rows)
  UPL: Processing...
[I 2026-02-12 11:35:49,696] A new study created in memory with name: no-name
-098885eb-58c0-4d57-8c1e-4ac2ca681717
OK (2430 rows)
  WIPRO: Processing... OK (2430 rows)

✓ Successfully processed 50 stocks
Combining datasets...

✓ Combined dataset shape: (2734, 400)
  Date range: 2012-12-03 to 2022-12-09
  Assets: 50 stocks
  Total observations: 2734

✓ Successfully loaded 2734 rows from local CSV

✓ Data Loaded Successfully
  Shape: (2734, 400)
  Date Range: 2012-12-03 to 2022-12-09
  Features: 400 (stocks × features)

================================================================================
STEP 2: DATASET STATISTICS
================================================================================
Number of assets in portfolio: 50
Total trading days: 2734
Average daily log return (all assets): -2.4447%

================================================================================
STEP 3: BAYESIAN HYPERPARAMETER OPTIMIZATION
================================================================================
Training data: 1640 days (2012-12-03 to 2018-11-14)
Validation data: 1094 days (2018-11-15 to 2022-12-09)
   >>> Starting Transformer Warm-up for 1 epochs...
   >>> Warm-up Complete.
[I 2026-02-12 11:35:56,442] Trial 0 finished with value: -0.7209548328616777
and parameters: {'lr': 1.147008011139252e-05, 'ent_coef': 0.0035900192125193
55, 'clip_range': 0.23082398389023967, 'alpha_turnover': 0.00394526474516819
6, 'beta_slippage': 0.13453802162570175, 'gamma_risk': 18.45733947047228}. B
est is trial 0 with value: -0.7209548328616777.
   >>> Starting Transformer Warm-up for 1 epochs...
   >>> Warm-up Complete.
[I 2026-02-12 11:36:02,876] Trial 1 finished with value: -0.2691181892487834
5 and parameters: {'lr': 2.482441525400857e-05, 'ent_coef': 0.04251765719104
6526, 'clip_range': 0.13441255541272296, 'alpha_turnover': 0.009180321687317
562, 'beta_slippage': 0.4508407843183884, 'gamma_risk': 9.904847198443633}.
Best is trial 1 with value: -0.26911818924878345.
   >>> Starting Transformer Warm-up for 1 epochs...
   >>> Warm-up Complete.
[I 2026-02-12 11:36:09,172] Trial 2 finished with value: -0.549659848745654
and parameters: {'lr': 8.280676558177028e-05, 'ent_coef': 0.0284182609641276
36, 'clip_range': 0.27900343763072705, 'alpha_turnover': 0.0083548357901412
7, 'beta_slippage': 0.37820212220949556, 'gamma_risk': 14.400424920884713}.
Best is trial 1 with value: -0.26911818924878345.
```

```
✓ Best Hyperparameters Found:
  lr: 0.000025
  ent_coef: 0.042518
  clip_range: 0.134413
  alpha_turnover: 0.009180
  beta_slippage: 0.450841
  gamma_risk: 9.904847

======================================================================
STEP 4: WALK-FORWARD OUT-OF-SAMPLE VALIDATION
======================================================================
>>> FOLD 1: Train [0:750] | Test [750:1000]
   >>> Starting Transformer Warm-up for 3 epochs...
   >>> Warm-up Complete.
>>> FOLD 2: Train [250:1000] | Test [1000:1250]
   >>> Starting Transformer Warm-up for 3 epochs...
   >>> Warm-up Complete.
>>> FOLD 3: Train [500:1250] | Test [1250:1500]
   >>> Starting Transformer Warm-up for 3 epochs...
   >>> Warm-up Complete.
>>> FOLD 4: Train [750:1500] | Test [1500:1750]
   >>> Starting Transformer Warm-up for 3 epochs...
   >>> Warm-up Complete.
>>> FOLD 5: Train [1000:1750] | Test [1750:2000]
   >>> Starting Transformer Warm-up for 3 epochs...
   >>> Warm-up Complete.
>>> FOLD 6: Train [1250:2000] | Test [2000:2250]
   >>> Starting Transformer Warm-up for 3 epochs...
   >>> Warm-up Complete.
>>> FOLD 7: Train [1500:2250] | Test [2250:2500]
   >>> Starting Transformer Warm-up for 3 epochs...
   >>> Warm-up Complete.

======================================================================
STEP 5: PERFORMANCE METRICS & STATISTICAL TESTS
======================================================================

Core Performance Metrics:
  Total Return:             290.29%
  Sharpe Ratio:              1.225
  Sortino Ratio:             1.677
  Max Drawdown:             -17.42%
  Prob. Sharpe (PSR):        1.000

Running Monte Carlo Permutation Test (1000 iterations)...
  Monte Carlo P-Value:      1.0000
  ⚠ Not statistically significant (p >= 0.05) - May be due to luck

======================================================================
STEP 6: GENERATING VISUALIZATIONS
======================================================================
```



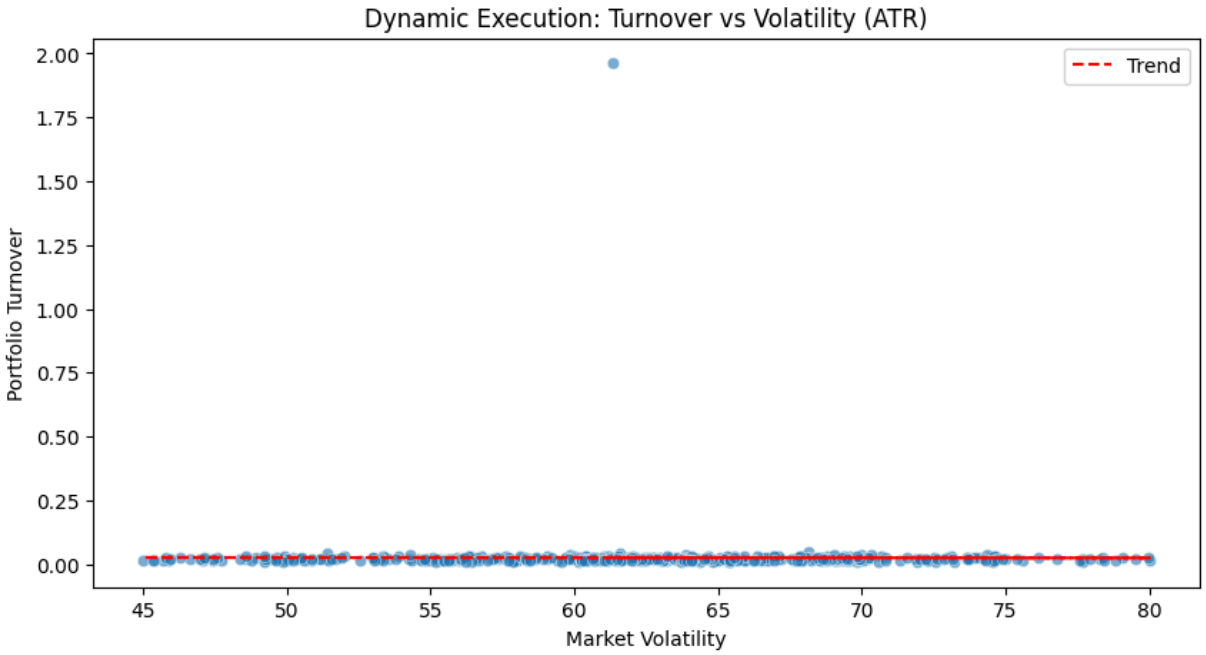




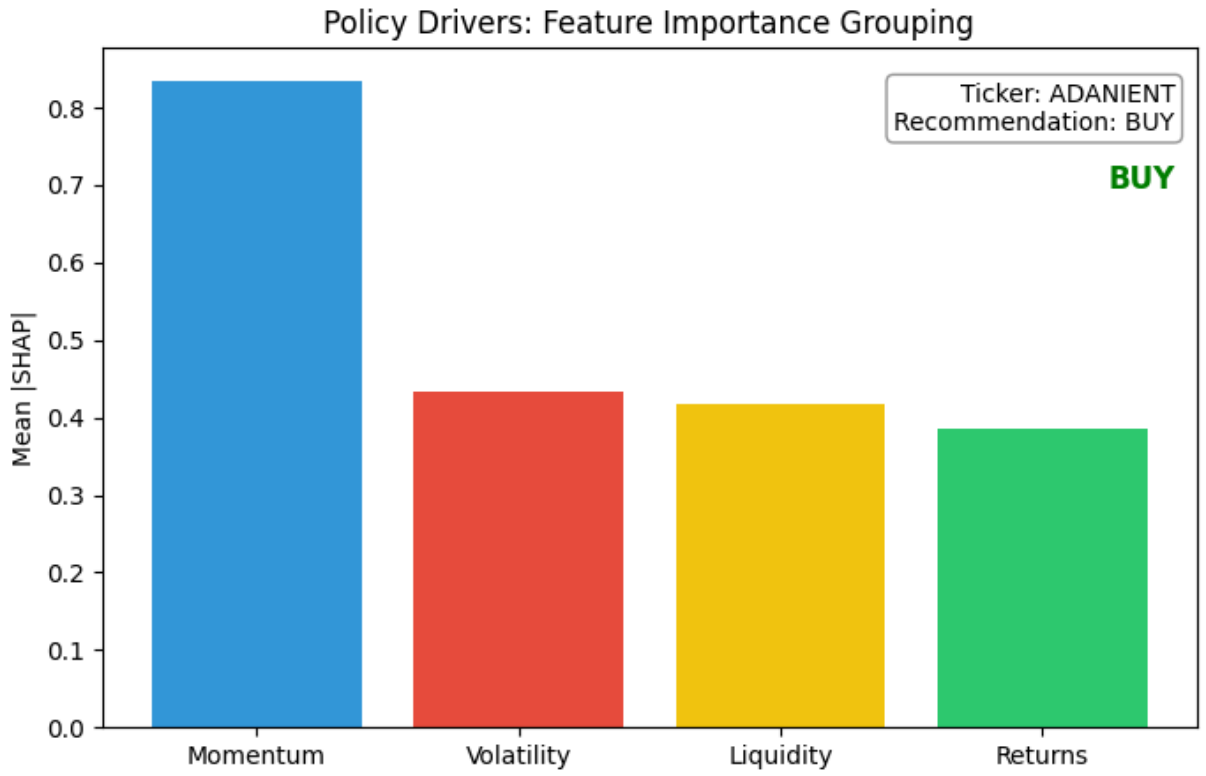
==========

STEP 7: EXPLAINABILITY ANALYSIS (SHAP VALUES)

==========




```
Calculating SHAP Values (this may take time)...
100%|██████████| 10/10 [34:01<00:00, 204.17s/it]
```




```
================================================================================
EXPERIMENT COMPLETE
================================================================================
```